

Application functionality.

Our project's core use case, Scheduled Automated Data Synchronization, is implemented and working. The system uses a scheduler to automatically trigger jobs for worker that discovers infrastructure assets with Ansible and then synchronizes that data with our Jira CMDB. Also, the conditions for UC-02 are implemented.

To run our project you need to have Linux machine/environment. To test it, run main.py file and test it via localhost:8000/docs.

CMDB API 0.1.0 OAS 3.1	
/openapi.json	
Jira Integration ^	
GET	/jira/assets Get Jira Assets v
GET	/jira/schemas Get Jira Schemas v
Jira Integration (Debug) ^	
GET	/jira/schemas Get Jira Schemas v
Assets ^	
GET	/assets/ Get All Assets v
Jobs ^	
GET	/jobs/ Get All Jobs v
GET	/jobs/{task_id} Get Job Status v
GET	/jobs/active Get Active Jobs v
POST	/jobs/{task_id}/cancel Cancel Job v
GET	/jobs/stats Get Job Stats v
Sync ^	

Here we can see the GET api call on the server asset in Jira named 'Tower of Chronia Test':

The screenshot shows the Jira REST Client interface. At the top, it indicates a GET request to `/jira/assets` with the description "Get Jira Assets". Below this, a note states "Get assets from Jira Asset Manager using AQL query." The "Parameters" section contains a table with two columns: "Name" and "Description". A single parameter is listed: `aql_query` (string, query) with the value `Name = "Tower of Chronia Test"`. Below the parameters is an "Execute" button and a "Clear" button. The "Responses" section shows the "Curl" command: `curl -X 'GET' \ 'http://127.0.0.1:8000/jira/assets?aql_query=Name%3D%22Tower%20of%20Chronia%20Test%22' \ -H 'accept: application/json'`. The "Request URL" is `http://127.0.0.1:8000/jira/assets?aql_query=Name%3D%22Tower%20of%20Chronia%20Test%22`. The "Server response" section shows a status code of 200 and a "Response body" containing a JSON array with one object:

```
[
  {
    "id": "2",
    "objectKey": "TA-2",
    "label": "Tower of Chronia Test",
    "attributes": [
      {
        "objectTypeAttributeId": "150",
        "objectAttributeValues": [
          {
            "displayValue": "Tower of Chronia Test",
            "searchValue": "Tower of Chronia Test",
            "referencedType": false,
            "value": "Tower of Chronia Test"
          }
        ]
      }
    ]
  }
]
```

For the last release, we plan to have a .bat file for running the whole project.

Testing

Initial project has test regarding the scheduler work, if it is assigning at least 1 task to worker, then the test is considered as passed. We use module called pytest:

```
(venv) philippaskov@Philips-MacBook-Pro UT_CMDB % pytest tests/test_scheduler.py
===== test session starts =====
platform darwin -- Python 3.11.9, pytest-8.4.2, pluggy-1.6.0
rootdir: /Users/philippaskov/Kool/semester5/softwareproject/UT_CMDB
plugins: anyio-4.11.0
collected 2 items

tests/test_scheduler.py .. [100%]

===== warnings summary =====
src/core/configs/config.py:7
/Users/philippaskov/Kool/semester5/softwareproject/UT_CMDB/src/core/configs/config.py:7: PydanticDeprecationWarning: Support for class-based `config` is deprecated, use ConfigDict instead. Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.12/migration/
  class Settings(BaseSettings):

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 2 passed, 1 warning in 0.07s =====
```

Continuous Integration (CI).

Continuous Integration is used throughout the process. Everyone has their own branch on Github and we are continuously committing and merging branches. Since we do not deploy our project somewhere, we do not use continuous deployment.

Continuous Integration is used in Github actions, the code is built and also tested there.

The screenshot shows the GitHub Actions interface for a workflow named 'update.yml file #23 #1'. The workflow is triggered by a push to the 'prod' branch. The status is 'Failure' with a total duration of 47s. The workflow file 'ci.yml' is shown, containing a single job named 'test' that failed after 39s. The annotations section shows an error message: 'test: Process completed with exit code 2.'

Triggered via	Status	Total duration	Artifacts
philip120 pushed -> 8c63015 prod	Failure	47s	-

ci.yml
on: push

```
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - run: test
```

Annotations
1 error

test
Process completed with exit code 2.

Detailed requirements.

Available in Github Wiki.

UI wireframes/mockups.

Jira Service Management (our UI) object “Tower or Chronia test” in schema “Test Ansible”:

Assets / Schemas

Test Ansible

SCHEMA TREE

Hardware Assets

Phones

Laptops

Servers

Ubuntu

Window...

Printers

Models

Hardware Mo...

Model Categ...

Software Mo...

1 object

Tower of Chronia Test

TA-2

Tower of Chronia Test

Details

Activity

Name	Tower of Chronia Test
CPU Temperature	35.0
CPU Model	test
Memory (GB)	43.0
Disk Usage	24.0
IP Address	test
Hostname	Test
Architecture	Test
Processor Model	None entered
Physical CPUs	None entered
Virtual CPUs	None entered

Scheduler work:

```
philippaskov@Philips-MacBook-Pro UT_CMDB % source venv/bin/activate
(venv) philippaskov@Philips-MacBook-Pro UT_CMDB % celery -A src.scheduler.celery_app beat --loglevel=info
Configuration file not found: /Users/philippaskov/Kool/semester5/softwareproject/UT_CMDB/src/core/config/config.json
Configuration file not found: /Users/philippaskov/Kool/semester5/softwareproject/UT_CMDB/src/core/config/config.json
celery beat v5.5.3 (immunity) is starting.
LocalTime -> 2025-10-19 20:49:17
Configuration ->
  . broker -> sqlalchemy+sqlite:///celery_broker.db
  . loader -> celery.loaders.app.AppLoader
  . scheduler -> celery.beat.PersistentScheduler
  . db -> celerybeat-schedule
  . logfile -> [stderr]@%INFO
  . maxinterval -> 5.00 minutes (300s)
[2025-10-19 20:49:17,753: INFO/MainProcess] beat: Starting...
[2025-10-19 20:49:17,775: INFO/MainProcess] Scheduler: Sending due task periodic-ansible-discovery (worker.tasks.discovery.discovery_task)
[2025-10-19 20:49:17,784: INFO/MainProcess] Scheduler: Sending due task periodic-linux-discovery (worker.tasks.discovery.discovery_by_type_task)
[2025-10-19 20:49:17,785: INFO/MainProcess] Scheduler: Sending due task periodic-auto-discovery (worker.tasks.discovery.auto_discovery_task)
```

Worker work:

```
(venv) philippaskov@Philips-MacBook-Pro UT_CMDB % celery -A src.worker.main worker --loglevel=info

----- celery@Philips-MacBook-Pro.local v5.5.3 (immunity)
--- ***** ---
-- ***** --- macOS-15.6.1-arm64-arm-64bit 2025-10-19 20:50:13
- *** --- * ---
- ** ----- [config]
- ** ----- .> app: cmdb_worker:0x107ebd9d0
- ** ----- .> transport: sqlalchemy+sqlite:///celery_broker.db
- ** ----- .> results: sqlite:///celery_results.db
- *** --- * --- .> concurrency: 2 (prefork)
-- ***** --- .> task events: OFF (enable -E to monitor tasks in this worker)
--- ***** ---
----- [queues]
      .> ansible exchange=ansible(direct) key=ansible

[tasks]
  . worker.tasks.discovery.auto_discovery_task
  . worker.tasks.discovery.batch_discovery_task
  . worker.tasks.discovery.discovery_by_type_task
  . worker.tasks.discovery.discovery_task
  . worker.tasks.sync_to_jira.sync_discovered_assets
  . worker.tasks.sync_to_jira.sync_task

[2025-10-19 20:50:13,995: INFO/MainProcess] Connected to sqlalchemy+sqlite:///celery_broker.db
[2025-10-19 20:50:14,001: INFO/MainProcess] celery@Philips-MacBook-Pro.local ready.
```

Scope

- **Iteration 3:**
 - More functionality
 - Add support for Windows and Solaris systems
 - Implement asset change tracking and audit logs

Expand test coverage and validation logic

- **Iteration 4:**

Finalization and Delivery

Optimize performance and security

Finalize documentation and architectural diagrams

Conduct user testing and gather feedback

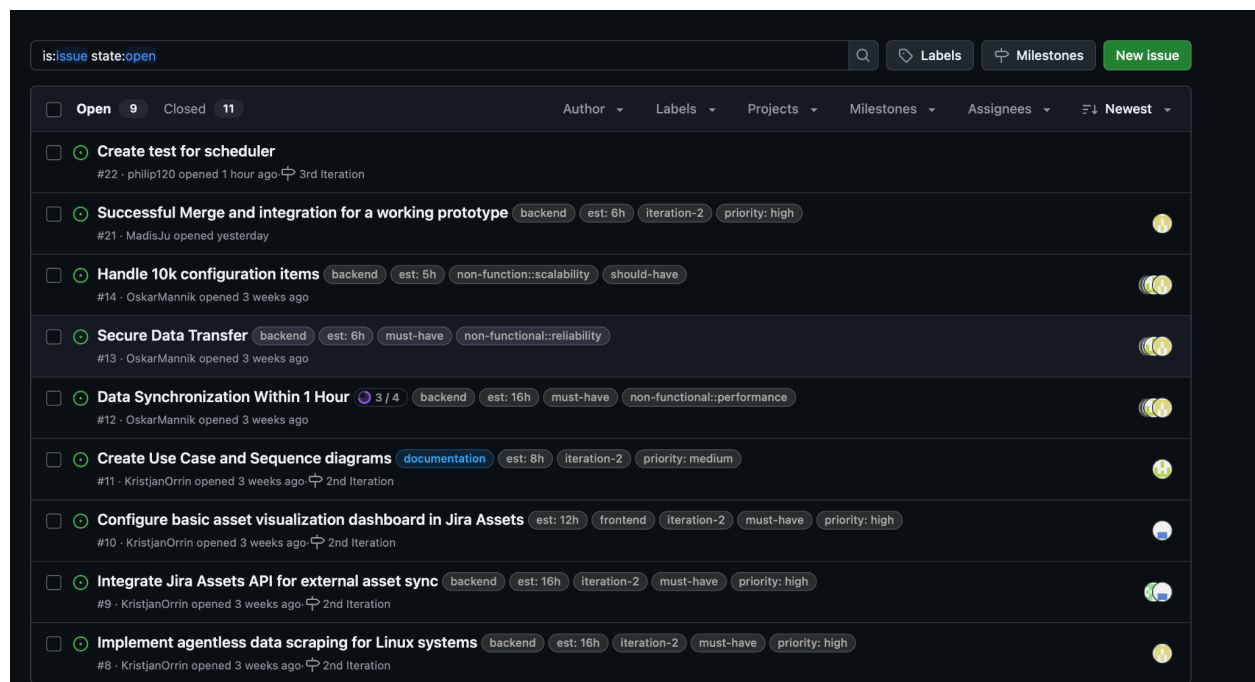
Prepare deployment package and handover materials

Release notes.

Available in Github. https://github.com/MadisJu/UT_CMDDB/releases/tag/release-2.0

Collaboration infrastructure is in use.

VCS is being used regularly and by all team members. Requirements in wiki or other documents are updated regularly and the issue tracking system is also updated and used regularly.



Individual contribution report

Madis - worker with core/ folder and overall logic of the project, connected different parts together

Philip - built Scheduler and its connections to other parts

Kristjan - built API part and worked with Jira

Oskar - built Worker and its connections to other parts